Blueprint Studio

Functional Specification of an Epistemic Authoring and Thinking Tool

---

0. Basic Definition

Blueprint Studio is not a text editor, not a note-taking tool, and not an LMS.

It is an **epistemic operating system**.

It does not serve the representation of thoughts, but their formalization, verification, and revision.

Its task is not to make writing easier, but to make thinking robust.

---

1. System Goal

Blueprint Studio forces the user to transform vague ideas into explicit, verifiable structures.

It should:

- Make implicit assumptions visible
- Enforce justification obligations
- Map logical dependencies
- Make contradictions detectable
- Document revisions traceably
- Make knowledge processes assessable

The learning product is not the result, but the revision history.

---

2. Epistemic Scaffolding (Structural Constraint)

The system is block-based, not page-based.

A blueprint can only be completed when all mandatory modules are formally valid:

Mandatory Modules:

- Question / Problem Definition
- Goal Definition
- Scope (Validity Range)
- Explicit Assumptions
- Concept Definitions
- Claim List

- Dependency Structure
- Tests / Counterexamples
- Uncertainty Zones
- Decision Proposal / Synthesis

Each block has formal requirements.

Example:

- A claim without status is invalid.
- An assumption without justification is inadmissible.
- A decision without dependency graph cannot be saved.

Blueprint Studio is not an advisor.

It is a compiler.

---

3. Visual Claim Management (Graph-Based Logic)

All central statements are modeled as claims, not as text paragraphs.

Each claim has:

- Type (empirical, logical, normative, operative, hypothetical)
- Status (secure, probable, uncertain, speculative)
- Scope
- Dependencies
- Test status

Claims are nodes in a directed graph.

Relations:

- supports
- contradicts
- requires
- specifies
- generalizes
- relativizes

---

4. Cascade Logic ("Tremor System")

When a claim is changed, the system must:

- Identify all dependent claims
- Automatically downgrade their status
- Mark conflicts

- Display revision needs

Didactic Effect:

The user sees that thinking has consequences.

This prevents rhetorical thinking.

---

5. Two AI Modes (Epistemic Role Separation)

5.1 Exploration Mode

Purpose: Idea generation

The AI may:

- Propose hypotheses
- Generate analogies
- Offer perspectives
- Create thinking variants

It may not:

- Set claims
- Assign status
- Make decisions

5.2 Precision Mode

Purpose: Structural verification

The AI checks:

- Concept ambiguities
- Implicit assumptions
- Logical breaks
- Scope errors
- Circular reasoning

It formulates no truths, only attacks.

---

6. Adversarial Mode (Systematic Stress Test)

The AI explicitly assumes the role of an opponent.

Attack Types:

- Counterexample generator

- Scope breaker
- Assumption detector
- Reversal argument
- Edge case simulation

The user must respond to every attack:

Options:

- Specify
- Restrict
- Defend
- Discard
- Restructure

Unanswered attacks block certifiability.

---

7. The Δ-Log (Epistemic Learning Biography)

Every change generates a mandatory log entry:

- What was changed?
- Why?
- By which attack?
- What cascade consequences?

The Δ-Log is not optional.

It is the assessment basis.

What counts is not the result, but the revision quality.

---

8. Assessment Logic

Assessed:

- Structural coherence
- Degree of explicitness
- Attack resistance
- Revision quality
- Status discipline
- Scope correctness

Not assessed:

- Opinion

- Style
- Rhetoric
- Worldview

---

9. Multi-User and Consolidation Mode

Blueprint Studio must enable group work:

- Parallel blueprint versions
- Claim comparison
- Conflict visualization
- Consolidation interfaces

The AI may here:

- Recognize conflicts
- Form clusters
- Display structural differences

It may not decide.

---

10. Output Formats

A blueprint is not a text.

It can be exported as:

- Certified knowledge graph
- Explanation view (for AI mediation)
- Didactics view
- Policy view
- Examination view

Each view accesses the same knowledge core.

---

11. Role in the Overall System

Blueprint Studio is:

- Knowledge generator
- Knowledge validator
- AI training source
- Didactics infrastructure
- Governance tool

It is the production site of epistemic units.

---

12. Guiding Principle

Blueprint Studio is not a writing tool.

It is a thinking instrument.

It does not make thinking easier, but robust.

Blueprint Studio

UI Specification v1.0

Epistemic Authoring and Thinking Tool

---

1. Basic Principles of the UI

Blueprint Studio is not an editor, but a working environment for thinking.

From this follow hard UI rules:

- **Structure before aesthetics** — the UI shows epistemic states, not beautiful texts.
- **States are visible** — uncertainty, conflict, incompleteness are marked.
- **No linear focus** — the graph is the document.
- **No silent failure** — errors must appear explicitly.
- **Every action is epistemic** — there are no purely cosmetic actions.

---

2. Overall Structure

The interface consists of four persistent areas:

```
┌──────────────────────────────────────────────────────────┐
│                      Menu Bar / Status                   │
├──────────────────┬──────────────────────────┬────────────┤
│ Structure        │ Epistemic Arena (Graph)  │ AI & Δ-Panel │
│ Panel (left)     │ (center)                 │ (right)    │
├──────────────────┴──────────────────────────┴────────────┤
│                Validation Bar / Compiler Status          │
└──────────────────────────────────────────────────────────┘
```

---

3. Structure Panel (left)

Function

The structure panel shows the formal architecture of the blueprint.

It is not a table of contents, but a completeness monitor.

3.1 Mandatory Blocks

Fixed list:

- Guiding Question
- Goal Definition
- Scope
- Assumptions
- Concepts
- Claim Network
- Dependencies
- Tests / Counterexamples
- Uncertainty Zones
- Synthesis / Decision

3.2 Status Display

Each block has a status:

- ❌ Invalid (missing / formally defective)
- ⚠️ Incomplete
- ✅ Formally valid
- 🔁 Revision needed
- 🔒 Locked (e.g., during consolidation)

3.3 Interaction

Click on block: → opens associated structure view → shows validation errors → shows dependencies

---

4. Epistemic Arena (Central Graph Editor)

4.1 Basic Definition

The arena is the actual document.

Everything else is views.

4.2 Claim Nodes

Each claim is an object with:

- ID (e.g., C1)
- Type (E, L, N, O, H)
- Status (S, W, U, X)
- Short content
- Scope
- Justification
- Tests
- History

4.3 Color Coding

Status: - Green = secure - Yellow = probable - Orange = uncertain - Red = collapsed / refuted - Gray = hypothetical

Type: - Empirical = Circle - Logical = Hexagon - Normative = Triangle - Operative = Rectangle - Hypothetical = dashed

4.4 Relations (Edges)

Types: - supports - contradicts - requires - qualifies - generalizes - exception-to

Edges are: - directed - typed - color-coded - clickable

4.5 Cascade Display

When a claim changes: - All dependent claims are color-marked - Status is automatically downgraded - Revision obligation is set

UI Elements: - “Show cascade” - “Show dependencies” - “Open revision list”

---

5. AI Panel (upper right)

5.1 Mode Switcher

Two hard modes:

- 🔍 Exploration Mode
- 🎯 Precision Mode
- ⚔️ Adversarial Mode

Never active simultaneously.

5.2 Exploration Mode

AI may: - Propose hypotheses - Generate analogies - Offer alternative models

UI: - Suggestions as material, not as claims - Drag & drop into graph possible - Always marked as “not validated”

5.3 Precision Mode

AI may: - Attack concepts - Mark scope problems - Find logical breaks - Show dependency errors

UI: - “Start structure check” - Error list - Direct linking into graph

5.4 Adversarial Mode

Here the AI is explicitly framed as opponent.

Display: “AI SYSTEM: Attack on Claim C3 started.”

Then: - Weak point - Attack type - Concrete counterexample

Buttons: - “Specify claim” - “Defend attack (justify)” - “Abandon claim”

Unanswered attacks block validation.

---

6. Δ-Log Panel (lower right)

6.1 Principle

The Δ-Log is not optional.

It is the learning biography.

6.2 Mandatory Fields per Entry

- Timestamp
- Affected claim
- Change type
- Justification
- Trigger (Peer, AI, self-correction)
- Cascade consequences

6.3 Visualization

- Timeline
- Filter (only AI, only Peer, only status changes)
- Comparison between versions

---

7. Validation Bar (bottom)

Blueprint Studio is a compiler.

Display: - ❌ Errors (blocking) - ⚠️ Warnings - **ℹ** Hints

Examples: - “Claim C4 has no scope” - “Assumption A2 is not used” - “Circular reasoning between C1 and C3”

---

8. Lock Logic

A blueprint can only be: - exported - submitted - certified

when: - no blocking errors - all mandatory blocks valid - all AI attacks answered - Δ-Log complete

---

9. Multi-User & Consolidation Mode

UI Functions: - Blueprint comparison (side-by-side graphs) - Conflict marking - Merge interface - Human decision fields - Consolidation protocol

---

10. Export Views

The knowledge core remains the same.

UI Views: - Explanation view - Policy view - Student view - Examination view - AI training view

---

11. UI Guiding Principle

Blueprint Studio is not a writing space.

It is an epistemic workbench.

It may be comfortable, but it must be correct.

Blueprint Studio — Data Model Specification v1.0

The data model is deliberately kept implementation-neutral. (JSON/GraphDB/SQL can be derived later.)

---

1. Basic Objects and Identities

1.1 Global IDs

All objects have: - `id` (stable, UUID or ULID) - `human_id` (short, readable, e.g., C1, A3, T7) - `created_at`, `updated_at` - `created_by` (User ID) - `version_id` (Snapshot/Revision, see Δ-Log)

Principle: `human_id` remains stable within a blueprint (C1 stays C1), even when content is revised.

---

2. Project and Blueprint Level

2.1 Project

Represents a working context (course, seminar, research project).

Fields: - `project_id` - `title` - `description` - `domain_tags` (e.g., "Climate Policy", "Medicine", "Economics") - `owners[]`, `members[]` (role model) - `default_language` - `policy_profile_id` (validation rules, see 9)

2.2 Blueprint

A blueprint is the central artifact.

Fields: - `blueprint_id` - `project_id` - `title` - `status` ∈ {draft, in_review, validated, certified, archived} - `certification` (optional, see 8) - `views_config` (optional: Explanation view etc.) - `active_snapshot_id`

Relationships: - has many `Snapshot` - has one `Graph` (Nodes/Edges) per Snapshot - has many `DeltaLogEntry`

---

3. Snapshot / Versioning

3.1 Snapshot

A snapshot is a complete state of the blueprint (immutable).

Fields: - `snapshot_id` - `blueprint_id` - `parent_snapshot_id` (for Fork/History) - `label` (e.g., R-03) - `reason` (short reason) - `created_at` - `created_by`

Principle: Every change generates either: - automatically a new snapshot (strict), or - a snapshot on "commit" (git-like).

For educational settings, I recommend "commit enforced" so the Δ-Log stays clean.

---

4. Knowledge Graph: Nodes

The graph consists of node types. All nodes share:

- `node_id`
- `human_id`
- `node_type`
- `title` (short)
- `content` (long)
- `scope_id` (optional, but mostly mandatory for claims)
- `tags[]`
- `attachments[]` (sources, files)

4.1 NodeType (Enum)

`QUESTION`, `GOAL`, `SCOPE`, `ASSUMPTION`, `TERM`, `CLAIM`, `TEST`, `EVIDENCE`, `DECISION`, `NOTE`

---

5. Claim Object (Core)

5.1 Claim (NodeType = CLAIM)

Fields: - `claim_id` (= node_id) - `claim_type` ∈ {E(empirical), L(logical), N(normative), O(operational), H(hypothetical)} - `claim_status` ∈ {S(secured), W(likely), U(uncertain), X(collapsed)} - `status_rationale` (justification for status) - `confidence_interval` (optional; numerical only when meaningful) - `assumption_refs[]` (IDs of assumptions explicitly needed) - `evidence_refs[]` (IDs of evidence) - `test_refs[]` (IDs of tests) - `risk_flags[]` ∈ {scope_drift, category_error, circularity, unfalsifiable, missing_evidence, ...} - `last_attack_state` (optional, see 7)

Rule: A claim without `claim_type` or `claim_status` is invalid.

---

6. Assumptions, Concepts, Scopes

6.1 Assumption (NodeType = ASSUMPTION)

Fields: - `assumption_id` - `assumption_type` ∈ {modeling, data, normative, operational} - `necessity` ∈ {critical, supporting} - `justification` (why assumed) - `violation_impact` (what happens if false)

6.2 Term (NodeType = TERM)

Fields: - `term_id` - `definition` - `disambiguations[]` (distinction from similar concepts) - `measurement_hint` (optional, for empirical terms)

6.3 Scope (NodeType = SCOPE)

Scope is its own node so that it is reusable and comparable.

Fields: - `scope_id` - `scope_type` ∈ {time, space, population, system_boundary, legal, method} - `parameters` (structured fields, e.g., time span, region) - `exclusions[]` (what explicitly does not apply) - `scope_tests[]` (typical drift traps)

---

7. Tests, Evidence, Attacks (Adversarial)

7.1 Test (NodeType = TEST)

Tests are formalized counterexamples / falsification attempts.

Fields: - `test_id` - `test_type` ∈ {counterexample, edge_case, stress_scenario, alternative_model, adversarial_question} - `target_claim_id` - `expected_failure_mode` (what would fail?) - `outcome` ∈ {open, passes, fails, inconclusive} - `resolution_note` (how it was responded to)

7.2 Evidence (NodeType = EVIDENCE)

Fields: - `evidence_id` - `evidence_type` ∈ {dataset, paper, expert_statement, measurement, legal_text, simulation} - `citation` (normalized) - `reliability` ∈ {high, medium, low, unknown} - `extraction` (what statement does it concretely support) - `verification_state` ∈ {unverified, verified, disputed}

7.3 AttackState (for AI/Peer Attacks)

Attacks are not tests alone; they are interaction events with mandatory response.

Fields: - `attack_id` - `source` ∈ {ai, peer, instructor} - `attack_vector` ∈ {scope_drift, hidden_assumption, weak_evidence, logical_gap, normative_smuggle, circularity} - `target_claim_id` - `attack_text` - `suggested_counterexample` (optional) - `required_response` ∈ {refine, defend, abandon} - `response_state` ∈ {open, responded, resolved} - `response_link` (DeltaLogEntry or Note)

Rule: Certification only when `response_state != open` for all attacks in the snapshot.

---

8. Certification and Consolidation

8.1 CertificationRecord

Fields: - `certification_id` - `blueprint_id` - `snapshot_id` (the certified state) - `certifying_body` (committee/chair/course) - `criteria_profile_id` - `date` - `valid_until` (optional) - `notes` - `signatures[]` (Reviewer IDs)

8.2 MergeRecord (Consolidation)

Fields: - `merge_id` - `source_blueprint_ids[]` - `target_blueprint_id` - `conflicts[]` (list, see 8.3) - `decisions[]` (justified decisions) - `created_by`, `created_at`

8.3 Conflict

Fields: - `conflict_id` - `conflict_type` ∈ {claim_contradiction, scope_mismatch, assumption_incompatibility, status_dispute, term_collision} - `involved_nodes[]` - `ai_detection_notes` (optional) - `human_resolution` (mandatory) - `resolution_snapshot_id`

---

9. Δ-Log (Revisions)

9.1 DeltaLogEntry

Fields: - `delta_id` - `blueprint_id` - `snapshot_id` - `timestamp` - `actor` ∈ {user, ai_suggested, peer_prompted, instructor} - `change_type` ∈ {add_node, edit_node, delete_node, add_edge, remove_edge, status_change, scope_change, merge_resolution} - `target_ids[]` (Nodes/Edges) - `before` (optional diff) - `after` (optional diff) - `reason` (mandatory, free text + tags) - `cascade_impacts[]` (IDs of affected claims + status changes)

Rule: No structure-relevant change without DeltaLogEntry.

---

10. Graph Edges

10.1 Edge

Fields: - `edge_id` - `snapshot_id` - `from_node_id` - `to_node_id` - `edge_type` ∈ {supports, requires, contradicts, qualifies, generalizes, exception_to, defines} - `strength` (optional, ordinal: low/med/high) - `notes` (optional)

---

11. Validation Rules (Policy Profiles)

11.1 PolicyProfile

So that the system can be differently strict in school/university/policy.

Fields: - `policy_profile_id` - `name` (e.g., “Upper Secondary”, “Bachelor”, “Policy Brief”) - `required_blocks[]` - `min_claims` - `must_have_tests_per_claim` (e.g., 1) - `allowed_claim_types[]` - `certification_rules` (e.g., all attacks resolved) - `export_permissions`

---

12. Minimal Validation Errors (Compiler Errors)

Error Types (blocking): - `missing_required_block` - `claim_missing_type_or_status` - `claim_without_scope` - `unresolved_attack` - `circular_dependency_detected` - `decision_without_supporting_claims` - `term_undefined_used_in_claim`

Warnings: - `unused_assumption` - `weak_evidence_low_reliability` - `scope_too_broad` - `untestable_claim_flagged`

---

13. MVP Data Model (Smallest Runnable Core)

For v1 MVP, the following are strictly required:

- Blueprint + Snapshot
- Claim Nodes + Scope + Assumptions
- Edges: supports / requires / contradicts
- DeltaLogEntry
- AttackState (minimal)
- Validation rules (minimal)

Everything else can be added iteratively.

Blueprint Studio — Technical Architecture v1.0

---

0. Architecture Principles

- **Graph is Source of Truth** — Flowing text is view, not data basis.
- **Immutable Snapshots** — Changes occur via snapshots + Δ-Log, not through silent overwriting.
- **Validation is Compiler** — No export/certification without "build green."
- **AI is Plugin, not Core** — AI may never deliver sole truth; only analysis/attacks/suggestions.
- **Reproducibility** — A certified snapshot must be identically reconstructable later.

---

1. Layer Model (High Level)

1.1 Client (Frontend)

- Graph Editor (Arena)
- Block Navigation (Mandatory Blocks)
- AI Panel (Modes)
- Δ-Log Viewer

- Validation Status / Error List
- Export Views

1.2 Application Layer (Backend API)

- Auth/Roles
- Blueprint CRUD
- Snapshot/Commit Service
- Graph Service (Nodes/Edges)
- Validation Service (Compiler)
- DeltaLog Service
- Review & Merge Service (Consolidation)
- Export Service (Views & Packages)

1.3 Knowledge Layer (Storage)

- Graph Storage (Nodes/Edges, versioned)
- Snapshot Storage (immutable)
- DeltaLog Storage
- Attachments/Artifacts (files, citations)

1.4 AI Adapter Layer (AI Services)

- Prompt/Mode Router
- Adversarial Generator
- Precision Checks
- Suggestion Generator
- Output Normalizer (AI → AttackState/Test)

1.5 Integration Layer (Optional)

- LMS (Moodle, Canvas)
- SSO (SAML/OAuth)
- Institutional Repository
- License/Contract API (later)

---

2. Frontend Architecture (Client)

2.1 Core Modules

Graph Editor - Node/Edge create/edit - Zoom/drag/select - Layout engine (auto + manual) - Highlighting (cascades)

Block Panel - Mandatory blocks status - Quick links to relevant nodes

Validation HUD - Errors/Warnings - Click-to-fix (jump to node)

AI Console - Mode toggle (Exploration/Precision/Adversarial) - Attack feed + actions

Delta Timeline - Revision list - Diff view (before/after)

2.2 Important UI Tech (MVP)

- A stable graph renderer (Canvas/WebGL/SVG — doesn't matter, main thing is performance)
- Client-side caching for Node/Edge data
- "Optimistic UI" with server commit confirmation

2.3 MVP Functionality (Client)

- Edit graph
- Display and answer attacks
- Create commit/snapshot
- Display validation errors
- Display Δ-Log

---

3. Backend Services (Application Layer)

3.1 Auth & Roles Service

Roles: - owner - editor - reviewer - certifier - viewer

MVP: OAuth + roles per project. **v2+:** SSO, course integration.

3.2 Blueprint Service

- `create_blueprint`
- `update_metadata`
- `change_status` (draft → in_review → validated → certified)

3.3 Snapshot / Commit Service

Purpose: Enforce immutable states.

Endpoints: - `create_snapshot(blueprint_id, parent_snapshot_id, reason)` - `commit_changes(snapshot_id, patchset)` - `set_active_snapshot(blueprint_id, snapshot_id)`

MVP Variant: Every "save" action = new snapshot (simple, robust).

Pro Variant: Working copy + explicit commit (git-like).

3.4 Graph Service

CRUD for Nodes/Edges always snapshot-bound.

- `add_node(snapshot_id, node)`
- `update_node(snapshot_id, node_id, patch)`
- `add_edge(snapshot_id, edge)`
- `remove_edge(snapshot_id, edge_id)`

Important: Update internally creates a new node version in the snapshot context.

3.5 Validation Service (Compiler)

Purpose: Block illegal states.

Input: `snapshot_id`

Output: - `errors[]` (blocking) - `warnings[]` - `hints[]` - `cascade_map` (which claims affected)

Validation rules come from PolicyProfile.

MVP: - Mandatory blocks present - Claim has type/status/scope - No open attacks - No circular reasoning (basic cycle detection) - Decision depends on claims

v2+: - Untestable heuristics - Scope drift patterns - Term collision detection

3.6 DeltaLog Service

Enforces Δ-entries on structural changes.

Mechanism: - Every graph operation requires `delta_reason` (UI enforces input). - Backend writes DeltaLogEntry + calculates `cascade_impacts`.

MVP: Manual reason **v2+:** Reason templates + auto tagging (AI-assisted, but not mandatory)

3.7 Review & Merge Service (Consolidation)

MVP: - Peer review: Comments + AttackState as "peer attack" - Merge: manual (editor decides, documents)

v2+: - Claim clustering (AI assisted) - Conflict list auto-generated - Merge UI with decisions per conflict

3.8 Export Service (Views)

Exports are "projections" of the graph.

MVP: - PDF/HTML export of current snapshot (Graph + Δ-Log + Blocks) - "Explanation view" as structured JSON export (for answer AI)

v2+: - SCORM/LMS packages - Interactive web embed - Multi-language view generation

---

4. Storage Architecture (Knowledge Layer)

4.1 Option A (MVP): Document Store + Versioning

- Snapshots as JSON bundles (Nodes/Edges + metadata)
- DeltaLog separate (append-only)
- Attachments in Object Storage

Advantage: Fast, simple, auditable. **Disadvantage:** Graph queries (e.g., dependency search) more expensive.

4.2 Option B (v2+): GraphDB + Snapshot Mechanism

- GraphDB for Nodes/Edges
- Snapshots as "named graphs" or "temporal versions"
- DeltaLog append-only

Advantage: Strong graph queries, consolidation easier. **Disadvantage:** Complexity.

Recommendation: MVP with Option A, then migration to GraphDB once consolidation scales.

---

5. AI Adapter Layer (Plugin)

5.1 Principle

AI may never directly change the graph.

It only produces: - Suggestions (Exploration) - AttackStates (Adversarial) - Warnings (Precision)

5.2 Services

Mode Router - Ensures: exactly one mode active

Prompt Builder - Builds prompt from Snapshot + Target nodes + PolicyProfile

Output Normalizer - AI output → structured AttackState/Test objects

Safety & Integrity Filter - Blocks "authority claims" ("this is true") - Enforces justification format

Citation Guard (optional) - Marks sources as unverified - Prevents "fake citations" through status "unverified"

5.3 MVP AI Function

- Adversarial: Counterexamples, scope drift, hidden assumptions
- Precision: Concept sharpening, logical gaps

v2+: - Claim clustering (for consolidation) - Translation views - Adaptive explanation engine

---

6. Cascade Engine (Propagation)

6.1 Goal

Changes to base claims should automatically mark effects.

Mechanism: - Directed dependency graph from requires/supports - On update: - Collect all downstream nodes - Apply status downgrade rules (policy-driven) - Generate UI highlight map - Fill Δ-Log `cascade_impacts`

MVP: Simple propagation + highlight **v2+:** Probabilistic propagation (only when meaningful), conflict resolution suggestions

---

7. Certification (Workflow + Technical)

7.1 Workflow

- `validated` means: formally clean (compiler green)
- `certified` means: human committee signs snapshot

Technical: - CertificationRecord binds snapshot hash. - Export contains hash + signatures.

MVP: Simple signer list **v2+:** Institutional signing, timestamping, audit trail

---

8. Integration (optional, but politically important)

8.1 LMS

- Course can use blueprints as assignment
- Play back grades/feedback

8.2 Repository

- Certified blueprints in institutional archive
- DOI optional

8.3 Licensing API (later)

- Access control to "certified blueprint packs"
- Usage rights, durations, audit logs

---

9. MVP Release Plan (technically minimal)

MVP must be able to:

- Create/edit graph (Claims, Edges)
- Validate mandatory blocks
- Enforce snapshot/Δ-Log
- Generate AI adversarial attacks + enforce response
- Export PDF/JSON
- Peer attack as AttackState

Not MVP:

- Automatic consolidation
- Complex GraphDB
- Licensing
- Multi-language explanation engine

---

10. Non-Functional Requirements (Minimum)

- **Auditability:** Every change traceable
- **Determinism:** Certified snapshot reproducible
- **Performance:** Graph with 500–2,000 nodes fluid
- **Offline capability (nice):** Local cache + later sync
- **Data protection:** Clear separation of personal data ↔ blueprint data

---

Blueprint Studio — MVP Tickets v1.0

A) Core Infrastructure

A1 — Project & Blueprint Basic Structure

User Story: As a user, I want to create a project and multiple blueprints within it, so I can organize thinking models.

Tasks: - Project CRUD - Blueprint CRUD - Blueprint Status: draft, in_review, validated

Acceptance Criteria: - User can create project - User can create, name, delete blueprint - Blueprint has status field - Blueprint saves metadata (title, description)

A2 — Snapshot System (immutable)

User Story: As a user, I want every structural change to be versioned, so learning remains traceable.

Tasks: - Snapshot data model - API: `create_snapshot()` - Every structural change requires snapshot or commit

Acceptance Criteria: - Every change creates new snapshot OR requires explicit commit - Old snapshots are read-only - Can switch between snapshots - Snapshot has label (e.g., R-03) and justification

A3 — Role Model (minimal)

User Story: As an instructor, I want to distinguish between editing, reviewing, and certifying.

Roles: - owner - editor - reviewer - viewer

Acceptance Criteria: - Editor can change graph - Reviewer can create attacks - Viewer read-only - Owner can change status

---

B) Graph Editor (Epistemic Arena)

B1 — Node Creation

User Story: As a user, I want to create nodes (Claim, Assumption, Scope…), so I can structure my model.

Node Types MVP: - CLAIM - ASSUMPTION - SCOPE - TERM - TEST

Acceptance Criteria: - User can create nodes - Node has: Title, Content - Node type is selectable - Nodes are visually distinguishable

B2 — Claim-Specific Fields

User Story: As a user, I must set type and status for claims, otherwise the claim is invalid.

Claim Fields: - Type: E, L, N, O, H - Status: S, W, U, X - Status justification (mandatory)

Acceptance Criteria: - Claim without type or status → validation error - UI enforces status justification - Status is visually coded

B3 — Edges (Relations)

User Story: As a user, I want to model relationships between statements.

Edge Types MVP: - supports - requires - contradicts

Acceptance Criteria: - User can draw edge - Type selectable - Direction visible - Type visually distinguishable

B4 — Basic Graph Navigation

Acceptance Criteria: - Zoom, pan, select - Focus on node - Highlight dependencies

---

C) Epistemic Scaffolding (Structural Constraint)

C1 — Mandatory Blocks Panel

Mandatory Blocks MVP: - Guiding Question - Scope - Assumptions - Claims - Tests - Synthesis

User Story: As a user, I want to see which parts of my blueprint are formally missing.

Acceptance Criteria: - Panel lists all mandatory blocks - Each block has status: ❌ ⚠️ ✅ - Click on block shows relevant nodes

C2 — Block Assignment

User Story: As a user, I must assign nodes to mandatory blocks.

Acceptance Criteria: - Each claim belongs to a block - Missing assignment = validation error

C3 — Lock Logic

User Story: I may not submit my blueprint if formal parts are missing.

Acceptance Criteria: - "Submit" button locked when: - Missing mandatory blocks - Claim without type/status - Claim without scope

---

D) AI Integration (Exploration / Precision / Adversarial)

D1 — AI Mode Switcher (hard exclusive)

User Story: As a user, I want to clearly switch between exploration, precision, and adversarial mode, so AI roles are not mixed.

Acceptance Criteria: - There is exactly one active mode (toggle, not multi-select) - Mode change is visibly logged in UI - Mode influences AI outputs (label + structured form)

D2 — Exploration Mode: Suggestions as "Material," not as Claims

User Story: As a user, I want to receive AI ideas without them automatically entering the model.

Acceptance Criteria: - AI outputs appear as "Suggestion Cards" - Each card has: Title, Text, "Adopt into Blueprint" button - Adoption creates a new node or fills a field, but only after user action - Adopted content is marked as "AI-suggested" (metadata)

D3 — Precision Mode: Structure Check (AI Hints)

User Story: As a user, I want AI to check my concepts/scopes/logic for weaknesses.

Acceptance Criteria: - "Start check" button - Result list with items (warning/hint) - Each item links to affected node(s) - Items are not blocking (that's what the compiler does), but can be converted to AttackStates

D4 — Adversarial Mode: Generate AttackState

User Story: As a user, I want AI to launch targeted attacks on a claim.

Acceptance Criteria: - User can select claim → "Start attack" - System generates AttackState with: - `attack_vector` (e.g., scope_drift / hidden_assumption / weak_evidence / logical_gap) - `target_claim_id` - `attack_text` - `required_response` (refine/defend/abandon) - `response_state=open` - AttackStates are displayed in UI as "open"

D5 — Response Obligation to Attacks (UI Workflow)

User Story: As a user, I must respond to every attack, otherwise I cannot submit.

Acceptance Criteria: - For every open attack, only these actions exist: - "Specify claim" → opens claim editor + requires change - "Defend attack" → requires justification (text + possibly new tests) - "Abandon claim" → status change to X + justification - After action, `response_state=responded` is set and a Δ-Log entry is created - Submit/Certify blocked if `response_state=open` exists

---

E) Δ-Log & Revision Biography

E1 — Δ-Log Mandatory for Structural Changes

User Story: As a user, I must justify every change, so learning remains traceable.

Acceptance Criteria: - On: add/edit/delete node, add/remove edge, status/scope change → UI requires "change reason" - Without reason, no save/commit - Δ-Log entry contains: - `change_type` - `target_ids` - `reason` - `actor` (user/ai_suggested/peer_prompted)

E2 — Δ-Log Timeline View

User Story: As an instructor, I want to see the thinking development.

Acceptance Criteria: - Timeline lists entries chronologically - Filter: only status changes / only AI attacks / only peer - Click on entry shows: affected nodes + (optional) before/after diff

E3 — Snapshot Labels (R-01, R-02 ...)

User Story: As a user, I want to clearly name revisions.

Acceptance Criteria: - Snapshot automatically creates label - User can add label/note - Export shows snapshot label + hash/ID

F) Validation (Compiler)

F1 — Minimal Compiler Service

User Story: As a user, I want to see a clear list of blocking errors.

Acceptance Criteria: - Validation delivers: - `errors[]`, `warnings[]` - At least the following blocking errors: - `missing_required_block` - `claim_missing_type_or_status` - `claim_without_scope` - `unresolved_attack` - `decision_without_supporting_claims` (if Decision included in MVP) - UI shows error count constantly at bottom (status bar)

F2 — Cycle Detection (basic circular reasoning)

User Story: As a user, I should recognize when my model is circular.

Acceptance Criteria: - Compiler detects simple cycles in requires/supports - Outputs error `circular_dependency_detected` - UI marks affected nodes/edges

F3 — Cascade Marking (minimal)

User Story: When I change a base claim, I want to see what is affected.

Acceptance Criteria: - When changing a claim, UI marks downstream nodes (1+ steps) - Δ-Log entry contains `cascade_impacts[]` (list of affected claim IDs) - (MVP: marking only; automatic status downgrade can be v2 if too risky.)

G) Export & Submission

G1 — Export PDF (Snapshot)

User Story: As a user, I want to export my blueprint as a documentable artifact.

Acceptance Criteria: - PDF contains: - Title, Project, Snapshot Label - Mandatory blocks (short text) - Claim list with type/status/scope - List of open/resolved attacks - Δ-Log summary (last N entries or complete optionally) - PDF references snapshot ID/hash

G2 — Export JSON ("Explanation View" / AI Feed)

User Story: As a system, I want a machine-readable export that serves as knowledge base.

Acceptance Criteria: - JSON contains: - nodes (Claims/Scopes/Assumptions/Terms/Tests) - edges - statuses - scope definitions - certification placeholder fields - No personal data in export (default)

G3 — Submission (in_review)

User Story: As a user, I want to submit my blueprint for review.

Acceptance Criteria: - Submit only possible when: - No blocking errors - No open attacks - Mandatory blocks complete - Status is set to `in_review` - After submission: editor rights optionally restricted (policy)

H) Review & Peer Attacks (minimal)

H1 — Reviewer Can Create AttackStates

User Story: As a peer reviewer, I want to leave attacks as structured units.

Acceptance Criteria: - Reviewer selects claim → "Peer attack" - Must choose `attack_vector` + enter text - AttackState is treated like AI attack (open/responded/resolved) - Response obligation applies identically

H2 — Comment/Note (lightweight)

User Story: As a reviewer, I want to add short comments without changing claim structure.

Acceptance Criteria: - Comments on nodes possible - Comments are optionally shown in export - Comments do not trigger validation errors

H3 — Validated Status (formal)

User Story: As an instructor, I want to mark a blueprint as formally valid.

Acceptance Criteria: - Only owner/instructor can set `validated` - Only when compiler green - Marking logged in Δ-Log

---

MVP Definition (final checklist)

Release v1 is achieved when:

- Graph editor works (Nodes/Edges)
- Mandatory blocks + validation block incorrect models
- Snapshots + Δ-Log enforce revision evidence
- AI can create attacks, user must respond
- Peer can create attacks
- Export PDF + JSON per snapshot
- Submission workflow (draft → in_review → validated)

Blueprint Studio — API Endpoints (MVP v1)

Conventions

- All write calls require `snapshot_id` or automatically create a new snapshot (depending on MVP variant).
- All write calls require `delta_reason` (mandatory).
- Responses always contain `active_snapshot_id` (if changed) and optionally `validation_summary`.

---

1) Auth & Roles

Method	Endpoint	Purpose
GET	`/me`	Current user, roles, projects
GET	`/projects/{project_id}/members`	List members
POST	`/projects/{project_id}/members`	Add member
PATCH	`/projects/{project_id}/members/{user_id}`	Update role
DELETE	`/projects/{project_id}/members/{user_id}`	Remove member

MVP minimal: add/remove, role set

---

2) Projects

Method	Endpoint	Purpose
POST	`/projects`	Create project
GET	`/projects`	List projects (for user)
GET	`/projects/{project_id}`	Project detail
PATCH	`/projects/{project_id}`	Update metadata
DELETE	`/projects/{project_id}`	Delete (owner only)

Body (minimal): `title`, `description?`

---

3) Blueprints

Method	Endpoint	Purpose
POST	`/projects/{project_id}/blueprints`	Create blueprint
GET	`/projects/{project_id}/blueprints`	List blueprints
GET	`/blueprints/{blueprint_id}`	Blueprint metadata + active_snapshot_id + status
PATCH	`/blueprints/{blueprint_id}`	Update title/description/policy_profile
POST	`/blueprints/{blueprint_id}/status`	Change status

Body: `title`, `description?`, `policy_profile_id?`

Status change body: `to_status` ∈ {draft, in_review, validated, certified}, `delta_reason`

Rules (server enforced): - `in_review` only when compiler green + no open attacks - `validated` only owner/instructor + compiler green

---

4) Snapshots / Versioning

Method	Endpoint	Purpose
POST	`/blueprints/{blueprint_id}/snapshots`	Create snapshot (explicit commit)
GET	`/blueprints/{blueprint_id}/snapshots`	List snapshots (labels, dates, authors)
GET	`/snapshots/{snapshot_id}`	Snapshot metadata + hash + created_by
POST	`/blueprints/{blueprint_id}/active-snapshot`	Set active snapshot (checkout)

Body: `parent_snapshot_id?` (default: active), `reason` (mandatory)

Returns: `snapshot_id`, `label`, `hash`

---

5) Graph: Nodes

Method	Endpoint	Purpose
GET	`/snapshots/{snapshot_id}/nodes`	List nodes (optionally filtered)
POST	`/snapshots/{snapshot_id}/nodes`	Create node
GET	`/snapshots/{snapshot_id}/nodes/{node_id}`	Node detail
PATCH	`/snapshots/{snapshot_id}/nodes/{node_id}`	Update node
DELETE	`/snapshots/{snapshot_id}/nodes/{node_id}`	Delete node

Query: `type?=CLAIM|SCOPE|ASSUMPTION|TERM|TEST`, `ids?=...`

Body: `node_type`, `title`, `content?`, `scope_id?`, type-specific fields (see Claim), `delta_reason` (mandatory)

Returns: `node_id`, `human_id`, updated snapshot info

Server checks: Deleting a node with dependent edges → either reject or cascade delete (MVP: reject with message)

---

6) Graph: Claims (convenience endpoints)

(Alternative: Claims are nodes; these endpoints are optional. I list them because teams like them.)

Method	Endpoint	Purpose
POST	`/snapshots/{snapshot_id}/claims`	Create claim
PATCH	`/snapshots/{snapshot_id}/claims/{claim_id}`	Update claim fields (incl. status change)

Body: `title`, `content`, `claim_type` (E/L/N/O/H), `claim_status` (S/W/U/X), `status_rationale`, `scope_id`, `delta_reason`

---

7) Graph: Edges

Method	Endpoint	Purpose
GET	`/snapshots/{snapshot_id}/edges`	List edges
POST	`/snapshots/{snapshot_id}/edges`	Create edge
DELETE	`/snapshots/{snapshot_id}/edges/{edge_id}`	Remove edge

Body: `from_node_id`, `to_node_id`, `edge_type` ∈ {supports, requires, contradicts}, `delta_reason`

---

8) Mandatory Blocks (Scaffolding)

Method	Endpoint	Purpose
GET	`/blueprints/{blueprint_id}/scaffold`	Get scaffold status
POST	`/snapshots/{snapshot_id}/scaffold/assign`	Assign node to block
POST	`/snapshots/{snapshot_id}/scaffold/unassign`	Unassign node from block

Returns: List of required blocks, mapping block → nodes, completion state per block

Body: `block_id` (e.g., SCOPE, CLAIMS, TESTS), `node_id`, `delta_reason`

---

9) Validation (Compiler)

Method	Endpoint	Purpose
POST	`/snapshots/{snapshot_id}/validate`	Run compiler validation
GET	`/snapshots/{snapshot_id}/validation`	Get last validation result (cache)

Returns: `errors[]` (type, message, related_node_ids), `warnings[]`, `hints[]`, `cascade_map?` (optional), `validation_hash` (optional)

---

10) Δ-Log

Method	Endpoint	Purpose
GET	`/blueprints/{blueprint_id}/delta-log`	Get delta log
GET	`/blueprints/{blueprint_id}/delta-log/{delta_id}`	Delta entry detail (diffs, impacts)

Query parameters: `snapshot_id?`, `change_type?`, `actor?`, `limit?`

(Δ-Log is automatically written server-side; UI only delivers `delta_reason`.)

---

11) AI: Suggestions & Attacks

Method	Endpoint	Purpose
POST	`/snapshots/{snapshot_id}/ai/suggest`	Exploration
POST	`/snapshots/{snapshot_id}/ai/precision-check`	Precision
POST	`/snapshots/{snapshot_id}/ai/attack`	Adversarial

Suggest body: `context_node_ids[]` (optional), `prompt_seed?`, `delta_reason?` (optional)
Returns: `suggestions[]` (cards: title, text, tags)

Precision-check body: `target_node_ids[]`, `check_types[]` (e.g., term_sharpness, scope, logic) **Returns:** `issues[]` (severity, node_ids, message)

Attack body: `target_claim_id`, `attack_vectors?` (optional preselect), `delta_reason?` (optional; the attack itself is logged) **Returns:** `attack_id`, `attack_vector`, `attack_text`, `required_response`, `response_state=open`

---

12) Attacks (AttackStates) — AI & Peer

Method	Endpoint	Purpose
GET	`/snapshots/{snapshot_id}/attacks`	List attacks (filter open/responded)
POST	`/snapshots/{snapshot_id}/attacks`	Create peer attack
POST	`/snapshots/{snapshot_id}/attacks/{attack_id}/respond`	Respond to attack
POST	`/snapshots/{snapshot_id}/attacks/{attack_id}/resolve`	Resolve (Reviewer/Instructor)

Create body: `target_claim_id`, `attack_vector`, `attack_text`, `required_response` (refine/defend/abandon), `delta_reason`

Respond body: `response_action` ∈ {refine, defend, abandon}, `response_text` (mandatory for defend), `linked_changes?` (node_ids changed) — optional, `delta_reason` (mandatory)

Server rules: Sets `response_state=responded`, triggers validate optionally

Resolve body: `resolution_note`, `delta_reason` (Optional in MVP — you can merge responded=resolved.)

---

13) Tests & Evidence (minimal optional)

Method	Endpoint	Purpose
POST	`/snapshots/{snapshot_id}/tests`	Create test (counterexample etc.)
PATCH	`/snapshots/{snapshot_id}/tests/{test_id}`	Update outcome

(Evidence can start MVP-simplified as attachment/link on the claim.)

---

14) Export

Method	Endpoint	Purpose
POST	`/snapshots/{snapshot_id}/export/pdf`	Export PDF

Method	Endpoint	Purpose
POST	`/snapshots/{snapshot_id}/export/json`	Export JSON
GET	`/exports/{export_id}`	Download/export status

Returns: File link / job id

JSON returns: JSON bundle (nodes/edges/scaffold/status/attacks)

(MVP: synchronous; v2: async jobs.)

---

15) Review Workflow (minimal)

Method	Endpoint	Purpose
POST	`/blueprints/{blueprint_id}/submit`	Alias for status → in_review + server checks
POST	`/blueprints/{blueprint_id}/validate-and-submit`	Convenience: run validate, if green → submit, else return errors

---

Blueprint Studio — Event Logic (MVP v1)

---

0) Concepts

- **Action:** UI interaction (user clicks, edits, draws edge…)
- **Domain Event:** Server-side event (NodeAdded, AttackOpened…)
- **Side Effect:** Automatically triggered consequence (Δ-Log, Validation, Cascade)
- **Hard Rule:** May never be skipped (compiler logic)
- **Soft Rule:** May happen later/optionally (comfort/performance)

---

1) Global Principle: "No silent change"

Every structural change must:

- Generate Δ-Log entry (Hard)
- Define a snapshot revision (auto or commit) (Hard)
- Update a validation (immediately or deferred) (Soft/Hard depending on context)

---

2) Change Flow (Standard)

2.1 Standard Flow for Structural Change

```
Action → Request (with delta_reason) → Apply Patch → Write DeltaLog →
Update Snapshot → Trigger Validation → Return validation_summary
```

Hard: - `delta_reason` missing → Request is rejected - Δ-Log write must succeed, otherwise rollback

Soft: - Validation can be deferred (e.g., after 1–2 seconds debounce), except for gate actions (submit)

---

3) Events by Functional Area

A) Nodes

A1 — Create Node

- **Action:** "Add node"
- **Domain Events:** NodeCreated, GraphChanged
- **Side Effects:**
  - Δ-Log: add_node (Hard)
  - (optional) Scaffold: Auto-assignment suggestion (Soft)
  - Validation: re-run (Soft)
  - UI: Node appears, status bar updated
- **Acceptance:** Node visible, Δ-Log contains entry, validation_summary updated.

A2 — Edit Node (Content / Fields)

- **Action:** "Edit node"
- **Domain Events:** NodeUpdated, GraphChanged
- **Side Effects:**
  - Δ-Log: edit_node (Hard)
  - Cascade analysis: Mark downstream nodes (Soft, but recommended)
  - Validation: debounced (Soft)
- **Hard Case:** Claim fields changed (Type/Status/Scope) → Cascade + Validation immediately (Hard-ish for consistency; at least update status bar)

A3 — Delete Node

- **Action:** "Delete node"
- **Domain Events:** NodeDeleted, GraphChanged
- **Side Effects:**
  - Δ-Log: delete_node (Hard)
  - Edge cleanup: Remove dependent edges or block (Hard)
    - MVP Recommendation: Block if edges exist → "first remove edges"
  - Validation: immediately (Hard-ish)

B) Edges

B1 — Create Edge

- **Action:** Draw edge + choose type
- **Domain Events:** EdgeCreated, GraphChanged
- **Side Effects:**
  - Δ-Log: add_edge (Hard)
  - Cycle-check trigger (only for requires/supports) (Soft, but warn UI immediately)
  - Validation: debounced (Soft)

B2 — Delete Edge

- **Action:** Remove edge
- **Domain Events:** EdgeDeleted, GraphChanged
- **Side Effects:**
  - Δ-Log: remove_edge (Hard)
  - Validation: debounced (Soft)

C) Scaffolding (Mandatory Blocks)

C1 — Assign Node to Block

- **Action:** "Assign to block"
- **Domain Events:** ScaffoldAssigned
- **Side Effects:**
  - Δ-Log: edit_node or scaffold_assign (Hard)
  - Validation: immediately (Hard-ish), because block completion is gate-relevant

C2 — Block Completion Recompute

- **Trigger:** On assign/unassign or block edits
- **Domain Event:** ScaffoldCompletionUpdated
- **Side Effects:**
  - Update UI status in left panel (Hard)
  - Update validation summary (Soft)

D) AI Modes & AI Outputs

D1 — Mode Switch

- **Action:** Toggle Exploration/Precision/Adversarial
- **Domain Event:** AIModeChanged
- **Side Effects:**
  - UI shows active mode (Hard)
  - Optional Δ-Log: mode_change (Soft, but useful for audit)
  - No validation needed

D2 — Exploration: Generate Suggestion

- **Action:** “Generate ideas”
- **Domain Event:** AISuggestionsGenerated
- **Side Effects:**
  - No graph change (Hard)
  - No Δ-Log obligation
  - Store suggestions as cards (optional server cache)

D3 — Adopt Suggestion

- **Action:** “Adopt”
- **Domain Events:** NodeCreated/NodeUpdated
- **Side Effects:**
  - Δ-Log: actor=ai_suggested (Hard)
  - Validation: debounced (Soft)

E) Adversarial / Attacks (AI & Peer)

E1 — Start Attack (AI)

- **Action:** “Start attack” on claim
- **Domain Event:** AttackOpened
- **Side Effects:**
  - Save AttackState, response_state=open (Hard)
  - Δ-Log: attack_created (Hard, minimal)
  - UI: Attack visible in panel, claim marked

E2 — Create Peer Attack

- **Action:** Reviewer “Peer attack”
- **Domain Event:** AttackOpened
- **Side Effects:** Like E1

E3 — Answer Attack (refine/defend/abandon)

- **Action:** “Specify claim” / “defend” / “abandon”
- **Domain Events:** AttackResponded, plus possibly NodeUpdated
- **Side Effects:**
  - Δ-Log: merge_resolution or edit_node + reason (Hard)
  - AttackState: response_state=responded (Hard)
  - If abandon: Claim status → X (Hard)
  - Validation: immediately (Hard) (because gate: unresolved_attack)

E4 — Close Attack (resolve)

- **Action:** Reviewer/Instructor “resolved”
- **Domain Event:** AttackResolved
- **Side Effects:** Optional, can coincide with responded (MVP)

F) Δ-Log

F1 — Create Δ-Log Entry

- **Trigger:** Every structural mutation
- **Domain Event:** DeltaLogWritten
- **Hard:** Fails → rollback mutation
- **Soft:** Diff computation (before/after) can be delayed

G) Validation (Compiler)

G1 — Trigger Validation

Soft Trigger (Debounced): - Node edit content - Edge add/remove - Scaffold assignment

Hard Trigger (Immediate): - Claim type/status/scope change - Attack respond - Submit/Status change - Snapshot finalize (commit)

G2 — Validation Result

- **Domain Event:** ValidationComputed
- **Side Effects:**
  - Bottom bar update (Hard)
  - Disable/enable submit button (Hard)
  - Mark nodes causing errors (Soft, recommended)

H) Snapshot/Commit

H1 — Auto-Snapshot (strict MVP option)

- **Trigger:** Every mutation
- **Domain Event:** SnapshotCreated
- **Side Effects:** Label increment, active_snapshot_id update

H2 — Commit Mode (alternative)

- **Trigger:** User clicks "Commit/Save revision"
- **Domain Event:** SnapshotCreated
- **Side Effects:** Requires dirty-state tracking

Recommendation for MVP: Auto-snapshot (robust, auditable).

I) Submit / Review Workflow

I1 — Submit (draft → in_review)

- **Action:** "Submit"
- **Hard Preconditions:**
  - Validation errors = 0
  - open attacks = 0
  - required blocks complete

- **Domain Event:** BlueprintSubmitted
- **Side Effects:**
  - Status update (Hard)
  - Optional: Lock editing or mark changes as new branch (Soft/MVP optional)

I2 — Set Validated

- **Action:** Instructor "validated"
- **Hard Preconditions:** errors=0
- **Domain Event:** BlueprintValidated

J) Export

J1 — PDF Export

- **Action:** Export PDF
- **Hard Preconditions:** None (but PDF must reference snapshot)
- **Domain Event:** ExportGenerated
- **Side Effects:** Store export artifact

J2 — JSON Export

Analogous

---

4) Gate Buttons (the "Compiler Gates")

These buttons are only active when:

- **Submit** → errors=0 AND open_attacks=0 AND blocks_complete
- **Validated** → errors=0 AND role=instructor
- **Certified (v2)** → certification workflow ok

---

5) MVP Performance Rules (so it doesn't lag)

- Validation debounced to 500–1500ms when typing
- Immediate validation only on gate events/attack respond
- Cascade highlight possible client-side (graph traversal in browser)
- Server delivers "source of truth" optionally

Sprint-0 Plan: Sequence + Risks

Goal of Sprint-0

A vertical "happy path" chain must work:

```
Create claim → Set relation → Generate attack → Respond →
Δ-Log + Snapshot → Validation green → PDF/JSON export
```

Everything else is luxury.

Phase 0.1 — Foundation (Day 1–3)

Deliverables: - Data models: Project, Blueprint, Snapshot, Node, Edge, AttackState, DeltaLogEntry - Auth + Roles minimal: owner/editor/reviewer/viewer - Storage decision (MVP): JSON Snapshot Bundles + DeltaLog append-only (simple)

Risks & Mitigation: - **R0:** Over-engineering (GraphDB too early) → MVP first as snapshot bundles, later migration. - **R1:** "Silent edits" (without Δ-Log) → Server rejects every mutation without delta_reason.

Phase 0.2 — Graph Core (Day 4–8)

Deliverables: - Graph API: nodes/edges CRUD snapshot-bound - Frontend graph minimal: Place nodes, draw edges, select/edit - Claim fields: type/status/scope + UI mandatory fields

Risks & Mitigation: - **R2:** Graph performance → Limit in MVP: 300–500 nodes fluid; optimize later. Layout: no heavy auto-layout as mandatory. - **R3:** Snapshot explosion (every change new snapshot) → MVP ok. But: Snapshots as delta patches or compress (server) is Sprint-1 topic, not now.

Phase 0.3 — Compiler/Validation (Day 9–12)

Deliverables: - Validation service with hard rules: - Claim missing type/status → error - Claim without scope → error - unresolved_attack → error - required blocks incomplete → error (MVP light) - Validation HUD at bottom + clickable error list

Risks & Mitigation: - **R4:** Validation loops / UI lag → Debounce on client; immediate only on submit/attack response. - **R5:** "Too strict" vs "too lax" → PolicyProfile minimal: Upper secondary/university later; MVP: hard core rules only.

Phase 0.4 — Attacks & Response Obligation (Day 13–17)

Deliverables: - AttackState CRUD - AI adversarial (initially also possible without real LLM as stub): - Generate attack_text + vector - Response workflow: - refine / defend / abandon - response_state open→responded - Creates Δ-Log + optional claim status change

Risks & Mitigation: - **R6:** AI makes "authority" ("this is true") → Output normalizer: AI may only deliver attack schemas, never truth judgments. - **R7:** Users can bypass attacks → Submit button gated: open_attacks block.

Phase 0.5 — Δ-Log & Snapshot Story (Day 18–20)

Deliverables: - Δ-Log timeline view + filter (minimal) - Snapshot list + checkout (read-only old states)

Risks & Mitigation: - **R8:** Δ-Log becomes annoying (user drop-off) → Reason templates (dropdown + free text) already in MVP: "Concept sharpened", "Scope reduced", "AI attack answered", "Evidence added". - **R9:** Unclear revisions → Every attack response automatically writes a clean reason.

Phase 0.6 — Export & Demo Readiness (Day 21–24)

Deliverables: - PDF export: Claim list + attacks + Δ-Log summary + snapshot ID - JSON export: nodes/edges/attacks/scaffold + snapshot hash - Submit: draft→in_review only when compiler green

Risks & Mitigation: - **R10:** PDF is only pretty, not verifiable → PDF must contain snapshot hash/ID (reproducibility). - **R11:** Data protection / personal reference → Export default without user data; only IDs/roleless.

Critical Decisions (that must be nailed down in Sprint-0)

- **Auto-snapshot vs Commit** — MVP: Auto-snapshot (robust, auditable)
- **Δ-Log mandatory** — Yes, but with templates.
- **Graph as source of truth** — Yes, text is view.
- **Gate buttons are sacred** — Submit blocked on errors/open attacks.

Minimal "Showcase" Flow (for Thomas/Advisory Board)

1. Create Claim C1 (E,W) + Scope
2. Create Claim C2, Edge supports
3. AI attack on C1 (scope drift)
4. User specifies Scope → Cascade marks C2
5. Δ-Log shows revision
6. Validation: green
7. PDF export → "certifiable artifact"

This is the demo that people immediately understand.

Blueprint Studio — Roadmap

Phase 1: MVP — "Didactically Real" (0–3 months)

Goal: The system must force real thinking, not just look pretty.

Core Functions: - ✓ Claim graph - ✓ Mandatory blocks - ✓ Δ-Log - ✓ Snapshots - ✓ Validation (Compiler) - ✓ Adversarial Mode (AI + Peer) - ✓ Export PDF/JSON - ✓ Review workflow

Pedagogical Goals: - Separation of opinion vs. justification - Revision obligation - Attackability of arguments - Visible thinking development

Metrics: - Can students build a formal model without guidance? - Can they correctly answer attacks? - Do genuine revisions emerge?

Result: A tool that slows down thinking but accelerates truth.

Phase 2: Academic Scaling (3–9 months)

Goal: From tool to academic operating system.

2.1 Consolidation Engine

Now your Alexandria/Synapse idea becomes real.

Functions: - Compare multiple blueprints on the same topic - Claim clustering - Conflict identification - Consolidation UI

Example: 20 students work on "CO_2 tax." → System recognizes: - 5 different scopes - 3 normative premises - 2 logically incompatible claims

These become explicit.

2.2 Certification Workflow

A blueprint can now be: - Draft - Validated (formal) - Certified (substantive)

With: - Committee - Signatures - Versioning - Validity period

This is the point at which your system becomes scientifically relevant.

2.3 Multi-View Generator

A certified blueprint can automatically generate: - Explanation for 12-year-olds - Explanation for bachelor students - Explanation for master students - Explanation for the public - Explanation for policymakers

All from the same model.

This is epistemically revolutionary.

Phase 3: Knowledge Economy (9–18 months)

3.1 Blueprint Market

Blueprints become: - Findable - Comparable - Licensable

Not as texts — but as structured knowledge artifacts.

3.2 Claim Licensing

AI companies don't buy texts.

They buy: - Structured claims - Verified justification networks - Clean scopes - Formal uncertainties

This is gold for training.

3.3 Framework Agreements with Universities

Universities deliver: - Certified knowledge models

AI companies deliver: - Infrastructure - Funding - Research grants

This prospectively replaces third-party funding.

3.4 "Answer AIs" as Consumer Product

Your thought: **AIs are only answer machines here**

Exactly.

They only explain what's in the blueprint — adapted to level and language.

Phase 4: Civilization-Capable Infrastructure (18+ months)

This sounds big, but it's logical.

4.1 Epistemic Hygiene as Infrastructure

Today: Opinions compete

In your system: **Models compete**

With: - Explicit assumptions - Formal attacks - Revision histories

4.2 Post-Text Society

Text is not abolished — but it becomes view.

The primary form is: **Graph + justification network + scopes + revisions**

4.3 Political Significance

The system can: - Formalize legislative justifications - Make policy simulations transparent - Expose lobby narratives - Structurally attack disinformation

Not morally. **Structurally.**

Why This Is Not an EdTech Product

EdTech = learn more efficiently

Blueprint Studio = learn to think differently

That is a fundamental difference.